Laboratorio 2 · Parte 2

Securización de la aplicación de análisis y gestión de riesgos

Asignatura de Análisis y Gestión de Riesgos · 4.º curso · Trabajo en el mismo grupo de la parte 1

1. Qué cambia

- **Nueva fecha de entrega del Laboratorio 2 completo (partes 1 y 2): viernes 30 de octubre de 2026, 23:59**, en la tarea «Entrega Laboratorio 2» de la PDU. Una entrega por grupo.
- Las sesiones del **lunes 26/10** y del **jueves 29/10** son de trabajo autónomo en grupo, sin profesor. Este documento contiene todo lo necesario para avanzar y finalizar el laboratorio.
- La parte 2 completa la misma aplicación de la parte 1: persistencia, evaluación y medidas, acceso protegido, validación, entornos y pruebas (prompts 7 a 13; el 14 es una ampliación opcional).
- **No basta con ejecutar los prompts.** Cada decisión de seguridad debe poder explicarse con vuestras palabras en `docs/requisitos-seguridad.md` y en el registro del grupo (apartado 5).

2. Plan de la semana

Cuándo	Laboratorio 2 (en grupo)
Lunes 26/10 (sesión)	Prompts 7, 8 y 9: base de datos, evaluación y medidas, acceso del gestor
Jueves 29/10 (sesión)	Prompts 10, 11, 12 y 13: validación, entornos, pruebas y entrega
Hasta el viernes 30/10, 23:59	Revisión final, registro del grupo, etiqueta y entrega

Dedicación orientativa: las dos sesiones (200 minutos) y unas 2 o 3 horas de trabajo autónomo por persona. El resto del tiempo autónomo de la semana es para los Ejercicios prácticos 1.

3. Normas (las mismas de la parte 1)

- Solo datos ficticios. No carguéis en la IA datos personales, documentación real, credenciales ni material restringido.
- **Nunca** escribáis contraseñas, tokens ni cadenas de conexión en el chat de la IA, en el código, en los documentos ni en capturas. Se custodian en Bitwarden (o en Vaultwarden, si hacéis la ampliación) y se configuran en `.env.local` (excluido de Git) o en las variables de entorno de Vercel.
- Revisad cada cambio que genere la IA antes de aceptarlo y guardad un commit al final de cada etapa. Cada integrante hace commits con su propia cuenta de GitHub: el historial es la evidencia de vuestro trabajo.
- Las pruebas se hacen solo sobre vuestro proyecto.

- Si exponéis una credencial por error: revocadla o cambiadla de inmediato y anotadlo en `docs/incidencias.md`. Borrarla del último archivo no elimina la exposición. Declarar el incidente se valora; ocultarlo, no.

4. Prompts de la aplicación

Usadlos en orden sobre el proyecto `Laboratorio2` de la parte 1. Al terminar cada uno, comprobad el resultado, revisad lo generado y haced commit con el mensaje indicado.

7. Base de datos y persistencia

> Trabaja sobre el proyecto existente Laboratorio2. Vamos a guardar los riesgos en una base de datos.
>
> 1. Usaremos PostgreSQL en el proveedor (por defecto, Neon añadido desde el Marketplace de Vercel, plan gratuito). Crear la base de datos y configurar las variables de entorno: no pidas valores ni los muestres.
> 2. Diseña una tabla `riesgos` con los campos del formulario (proceso, activo, objetivo, condición, amenaza, evento, consecuencia, supuestos) y además: probabilidad, impacto, justificación de cada uno, puntuación, nivel, medida, prioridad, comprobación de eficacia, estado de la medida, fecha de creación y de modificación. Crea la migración en `db/` y explica cómo aplicarla.
> 3. Prepara `db/roles.sql` para crear **dos cuentas**: una administradora, solo para migraciones, y otra para la aplicación con los permisos mínimos sobre la tabla `riesgos` (consultar, insertar, modificar y borrar, sin crear ni borrar tablas). No incluyas contraseñas en el archivo.
> 4. La aplicación leerá la conexión de la variable `DATABASE_URL` (cuenta de la aplicación). La de administración, `DATABASE_URL_ADMIN`, solo se usará en local para migrar. Actualiza `.env.example` con los nombres, sin valores, y explica la finalidad de cada uno.
> 5. Crea en el servidor las operaciones para listar, crear y modificar riesgos. El navegador nunca debe recibir la cadena de conexión.
> 6. Sustituye el aviso «Borrador temporal» por un listado de riesgos guardados que se mantiene al recargar la página.
> 7. Documenta en `docs/base-de-datos.md` el modelo, las dos cuentas y cómo se aplica la migración, sin valores sensibles.
>
> Al terminar, dime qué tengo que hacer yo en el proveedor y en Vercel, y cómo compruebo que un riesgo se conserva al recargar. Commit: «Añade persistencia de riesgos».

8. Evaluación, nivel y medida

> Añade al formulario la evaluación y la medida, siguiendo `docs/evaluacion-riesgos.md`.
>
> 1. Probabilidad e impacto de 1 a 3, cada uno con su descriptor visible y una justificación obligatoria. La falta de información se registra como supuesto o información pendiente, no como probabilidad alta.
> 2. La puntuación (probabilidad × impacto) y el nivel (bajo 1–2, medio 3–4, alto 6–9) se calculan **en el servidor**, en una función única y probada; el navegador solo los muestra.

3. Medida asociada: descripción concreta, prioridad, efecto esperado, forma de comprobar su eficacia y estado (propuesta, implantada, verificada). Una medida propuesta no cambia la valoración.
4. Dos vistas: «Resumen para dirección» (riesgo en una frase, nivel, objetivo afectado y medida prioritaria) y «Detalle técnico».

Al terminar, muestra las nueve combinaciones de probabilidad e impacto con su nivel. Commit: «Añade evaluación y medidas».

9. Acceso del gestor y autorización en el servidor

Vamos a proteger el acceso a los registros y las operaciones de escritura.

1. Implementa un acceso sencillo para un único perfil «gestor». Si propones una biblioteca, que esté mantenida, y explica por qué la eliges. La contraseña nunca se guarda en claro: solo su resumen (hash) en una variable de entorno o en la base de datos. La sesión se guarda en una cookie `HttpOnly`, `Secure` y `SameSite`, firmada con un secreto de servidor.
2. Sin sesión válida no se pueden ver los registros ni crear, modificar o borrar. **Cada operación de escritura comprueba la sesión en el servidor**, aunque la interfaz oculte los botones.
3. Añade un cierre de sesión y un límite razonable de intentos fallidos.
4. Añade a `.env.example` los nombres de los nuevos secretos, sin valores, y explica cómo generarlos.

No me pidas la contraseña ni la muestres: yo generaré el valor, lo guardaré en Bitwarden y lo configuraré. Al terminar, explica cómo comprobar que una escritura sin sesión se rechaza. Commit: «Protege acceso y escrituras».

10. Validación de entradas y errores

1. Valida en el servidor todas las entradas: tipos, longitudes máximas, campos obligatorios y valores permitidos (activo, objetivo, probabilidad, impacto, prioridad y estado). Rechaza lo que no cumpla con un mensaje claro.
2. Los errores públicos no deben mostrar trazas, consultas, rutas internas ni credenciales. Registra en el servidor lo necesario para diagnosticar, sin secretos.
3. Trata todos los textos como texto plano al mostrarlos.
4. Actualiza `docs/requisitos-seguridad.md`: SEG-05 y SEG-07 con implementación, prueba y evidencia.

Commit: «Valida entradas y controla errores».

11. Entornos separados

1. Comprueba que desarrollo (local) y Preview usan bases de datos o ramas distintas y secretos distintos. Si usas Neon desde Vercel, explica si las ramas de Preview están activadas y qué datos contienen.
2. Configura en Vercel las variables solo en los entornos que las necesitan. No uses `--prod`.
3. Documenta en `docs/gestion-secretos.md`, sin valores: cada secreto, su finalidad, dónde se usa, en qué entorno, dónde se custodia (Bitwarden) y cómo se cambia si se expone.

Commit: «Separa entornos y documenta secretos».

12. Pruebas

> Crea pruebas automáticas y un informe en `docs/pruebas.md` con fecha, commit y resultado de cada una:
>
> 1. **Clasificación:** las nueve combinaciones de probabilidad e impacto devuelven la puntuación y el nivel correctos.
> 2. **Validación:** se rechazan un campo obligatorio vacío, un texto demasiado largo y un valor fuera de la lista.
> 3. **Persistencia:** un riesgo creado sigue disponible tras recargar.
> 4. **Escritura no autorizada:** crear o modificar sin sesión devuelve un rechazo (401 o 403) y no cambia la base de datos.
> 5. **Secretos:** revisión del historial de Git en busca de secretos (con una herramienta de detección, si está disponible, o con una búsqueda documentada) y comprobación de que el código enviado al navegador no contiene valores de servidor.
>
> Ejecuta las pruebas y muestra el resultado. Si alguna falla, no la ocultes: explica la causa. Commit: «Añade pruebas de seguridad y funcionamiento».

13. Despliegue y entrega

> 1. Publica un nuevo despliegue Preview y comprueba en él las pruebas 3 y 4.
> 2. Actualiza `docs/despliegue.md` (URL, fecha, commit, comprobaciones) y `docs/requisitos-seguridad.md` con el estado final de SEG-01 a SEG-08: cumplido, parcial o pendiente, con su evidencia.
> 3. Revisa que `README.md` explica el objetivo, la ejecución, el despliegue y cómo comprobar la entrega, y enlaza todos los documentos de `docs/`.
> 4. Crea la etiqueta Git `entrega-l2` sobre el commit final y dime su identificador.
>
> Commit: «Prepara la entrega del laboratorio 2».

14. Ampliación opcional: Vaultwarden local con Docker

> Prepara un entorno local de Vaultwarden en `vaultwarden/` para custodiar los secretos del proyecto.
>
> 1. `docker-compose.yml` con la imagen `vaultwarden/server` en una versión fijada (no `latest`), publicada solo en `127.0.0.1` y con los datos en un volumen local excluido de Git.
> 2. `SIGNUPS_ALLOWED` en `true` solo para crear la primera cuenta y después en `false`. Si se activa el panel de administración, `ADMIN_TOKEN` debe ser un resumen Argon2 generado con la propia imagen, guardado en un archivo de entorno excluido de Git (en `docker-compose.yml` hay que duplicar cada `$`).
> 3. Accede por `http://localhost`: el cliente web necesita un contexto seguro y `localhost` lo es; por una IP o un nombre sin HTTPS, el inicio de sesión fallará.
> 4. Documenta en `docs/vaultwarden.md` la instalación, la diferencia entre custodiar un secreto y configurarlo en la aplicación, y cómo se haría una copia de seguridad del volumen. No guardes en él contraseñas personales reales.
>
> Commit: «Añade entorno local de Vaultwarden».

5. Registro del grupo (`docs/registro-grupo.md`)

1. Quién hizo qué, con referencia a sus commits.
2. **Una decisión de la IA revisada o corregida por cada integrante**: qué propuso la IA, qué cambiasteis y por qué, con vuestras palabras.
3. Respuesta razonada a la pregunta de cierre: si dirección solo pudiera aprobar una medida, ¿cuál y cómo comprobaríais que protege el objetivo afectado?

6. Qué se entrega (viernes 30/10, 23:59)

En la tarea «Entrega Laboratorio 2» de la PDU, un integrante sube:

1. Un texto con: nombre y apellidos de todos los integrantes, URL del despliegue Preview, URL del repositorio, identificador del commit de la etiqueta `entrega-l2` y quién realiza la entrega.
2. Un PDF con `docs/requisitos-seguridad.md`, `docs/pruebas.md` y `docs/registro-grupo.md`.

En el repositorio (privado, con acceso concedido al profesor):

- `README.md`, y en `docs/`: `fuentes.md`, `requisitos-seguridad.md` (estado final), `caso.md`, `evaluacion-riesgos.md`, `formulacion-riesgo.md`, `base-de-datos.md`, `gestion-secretos.md`, `pruebas.md`, `despliegue.md`, `registro-grupo.md` e `incidencias.md` (aunque esté vacío). `vaultwarden.md` si hacéis la ampliación.
- `.env.example` sin valores y ningún secreto en archivos ni en el historial.
- Historial completo, sin reescribirlo, con commits de todos los integrantes.